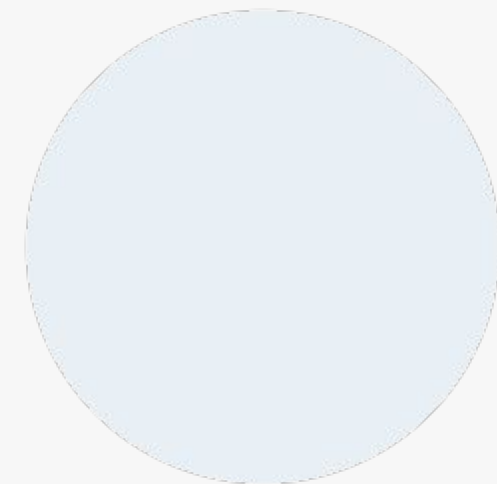


ASTON.



Репортеры



astondevs.ru

Понятие и назначение



Результатом работы (сессии тестирования) инженера считается отчёт. Репортинг - построение читаемого отчета о результатах выполнения тестов. В отчете прежде всего должна присутствовать информация о:

Количестве запущенных тестов

По каждому тесту:

- его название
- название сьюта
- результат
- время выполнения
- сообщение об ошибке, если тест упал

Количество успешно выполненных тестов

Количество упавших тестов

Время выполнения всех тестов

Дата и время запуска

Понятие и назначение



Эту информацию должен содержать любой отчет о выполнении тестов. Также отчет может содержать дополнительную информацию, полезную для дебаггинга:

- Полный стек трейс, если тест упал
- Последовательность шагов (test story) и их результат
- Встроенные скриншоты на каждый шаг и/или при падении теста
- Встроенные логи и/или другие attachments (любые текстовые и медиа файлы, включая html dump страницы)
- Значения переменных при DDT
- Значения переменных тестового окружения (версия приложения, браузер и другое)

Понятие и назначение



Кроме этого отчет может содержать отчеты о покрытии требований и другую визуальную информацию:

- Чарты и графики о результатах выполнения тестов
- Чарты и графики о покрытии требований
- Другая визуальная информация сводного характера

Информация о результатах теста получается и сохраняется после запуска тестов тест раннером. Часто это xml файл. Для тестовых фреймворков xml файл с результатами имеет один и тот же формат, что позволяет инструментам, строящим визуализированный отчет (чаще всего html), работать с ним единообразно.

Понятие и назначение



Таким образом, информация о результатах прохождения тестов собирается и сохраняется при запуске тестов, но визуализироваться может на следующих этапах:

- Сразу после запуска непосредственным тест раннером (например, testng или junit)
- При запуске билд инструментом (surefire plugin для maven)
- При запуске через CI (соответствующие плагины)
- Отдельный инструмент для генерации отчета (вручную)

Понятие и назначение



Кроме того, если набор информации, которую собирает тест раннер, вам кажется неполным, то можно использовать различные test report фреймворки, которые собирают больше информации и строят более красивые и полные отчеты, либо создать такой инструмент самому. Как правило, такие инструменты состоят из 3 компонентов:

- Адаптер для тест раннера (listener). Этот модуль подключается к тест раннеру (непосредственному или встроенному в билд инструмент) и собирает информацию о выполнении тестов независимо от встроенного в тест раннер механизма.
- Модель - это информация о формате хранения собранной информации о выполнении тестов (структура xml файла, например)
- Генератор html репорта на основе имеющейся модели по информации, сохраненной в конкретном xml файле.

Понятие и назначение



Отчет должен быть составлен в такой форме, что любое заинтересованное лицо, далекое от разработки ПО (например, заказчик), могло его понять и проанализировать, сделать нужные выводы, например, об изменении качества в динамике. До сегодняшнего дня мы запускали автотесты в IDE и консоли и получали результат в виде логов. Это прекрасно, т. к логи — замечательный инструмент. Читая логи, часто можно понять, что пошло не так. Но отчёты о тестировании читаем не только мы. Заинтересованные лица хотят получать отчёты вне зависимости от того, как именно протестирован продукт — вручную или автоматически. Важен результат, а не инструмент.

Allure Report



Allure — фреймворк с открытым исходным кодом. Этот фреймворк разработан компанией «Яндекс» специально, чтобы избавить инженеров по автоматизации тестирования от «боли» составления красивых отчётов о прохождении автотестов. Ссылка на статью с подробным объяснением механики работы фреймворка находится в секции «Дополнительные материалы».

Подключение к проекту

Код для подключения к проекту Allure (на примере JUnit 5) находится на странице официальной документации.

https://docs.qameta.io/allure/#_junit_5

Использование аннотаций



Достоинство фреймворка заключается в том, что для его использования не надо изменять действующий код. Достаточно аннотировать нужные классы и методы специальными маркерами: наиболее употребимые перечислены в таблице ниже:

DisplayName	Добавление человеко-читаемого названия тестовому методу
Description	Добавление описания к тестовому методу
Attachment	Добавление вложения в отчёт
Link / Issue / TmsLink	Добавление ссылки на систему менеджмента тест-кейсами / баг-трекер
Severity	Добавление поля Severity для тестового метода в отчёт
Epic / Feature / Story	Разделение тестовых методов или классов на функционал-специфичные группы
Step	Название шага тест-кейса. Обычно применяется к методам Page Object классов

Получение отчётов



Чтобы получить отчёт, надо:

- открыть терминал;
- открыть папку проекта;

ввести команду:

`allure serve allure-results,`

где `allure-results` — папка с результатами тестирования. Затем на локалхосте запустится сервер Allure, и откроется отчёт в веб-форме.

Протоколирование действий



Мы познакомились с пакетом `org.openqa.selenium.support`, но познакомились не со всеми его классами. В этом уроке обратим внимание на следующие два представителя, позволяющие отслеживать и протоколировать действия драйвера во время тестов. Первый — интерфейс `WebDriverEventListener`, предоставляющий событийные методы.

Методы имеют понятную семантику:

```
void beforeAlertAccept(WebDriver driver);
void afterAlertAccept(WebDriver driver);
void afterAlertDismiss(WebDriver driver);
void beforeAlertDismiss(WebDriver driver);
void beforeNavigateTo(String url, WebDriver driver);
void afterNavigateTo(String url, WebDriver driver);
void beforeNavigateBack(WebDriver driver);
void afterNavigateBack(WebDriver driver);
void beforeNavigateForward(WebDriver driver);
void afterNavigateForward(WebDriver driver);
void beforeNavigateRefresh(WebDriver driver);
void afterNavigateRefresh(WebDriver driver);
void beforeFindBy(By by, WebElement element, WebDriver driver);
void afterFindBy(By by, WebElement element, WebDriver driver);
void beforeClickOn(WebElement element, WebDriver driver);
void afterClickOn(WebElement element, WebDriver driver);
void beforeChangeValueOf(WebElement element, WebDriver driver,
    CharSequence[] keysToSend);
void afterChangeValueOf(WebElement element, WebDriver driver,
    CharSequence[] keysToSend);
void beforeScript(String script, WebDriver driver);
void afterScript(String script, WebDriver driver);
void beforeSwitchToWindow(String windowName, WebDriver driver);
void afterSwitchToWindow(String windowName, WebDriver driver);
void onException(Throwable throwable, WebDriver driver);
<X> void beforeGetScreenshotAs(OutputType<X> target);
<X> void afterGetScreenshotAs(OutputType<X> target, X screenshot);
void beforeGetText(WebElement element, WebDriver driver);
void afterGetText(WebElement element, WebDriver driver, String text);
```

Протоколирование действий



Второй — `EventFiringWebDriver` — javadoc класса говорит, что это обёртка над экземпляром `WebDriver`, позволяющая регистрировать `WebDriverEventListener` для логирования. Выглядит «оборачивание» следующим образом:

```
@BeforeEach
public void setUp() {
    EventFiringWebDriver driver = new
    EventFiringWebDriver(new ChromeDriver());
    driver.register(new CustomEventListener());
}
```

В качестве экземпляра драйвера воспользуемся `EventFiringWebDriver`, конструктор которого принимает экземпляр «настоящего» драйвера. Далее для протоколирования действий драйвера регистрируем собственный «слушатель событий», реализующий интерфейс `WebDriverEventListener`. В чём именно будет заключаться протоколирование, решает тестировщик в зависимости от своих пожеланий:

- в выводе сообщения в консоль;
- в сохранении сообщений в специальный log-файл.

Сохранение скриншота при получении



ИСКЛЮЧЕНИЯ

Стоит обратить внимание на возможность получения и сохранения скриншотов веб-страниц, которые создаются в процессе выполнения теста. Сохранять скриншот для последующего анализа можно постоянно, но наиболее логично будет их сохранение при получении исключений драйвера. Например, когда драйвер не находит элемент по указанному ID. Интерфейс `WebDriverEventListener` имеет подходящий метод — `onException`, в него мы и поместим подходящую нам логику. Но для начала в пакете утильных (вспомогательных) классов создадим класс `ScreenshotMaker` со статическим методом `makeScreenshot`:

```
public class ScreenshotMaker {  
    public static void makeScreenshot(WebDriver driver, String  
filename) {  
        File temp = ((TakesScreenshot)  
driver).getScreenshotAs(OutputType.FILE);  
        File destination = new File("./target/" + filename);  
        try {  
            FileUtils.copyFile(temp, destination);  
        } catch (IOException exception) {  
            exception.printStackTrace();  
        }  
    }  
}
```


Сохранение скриншота при получении



ИСКЛЮЧЕНИЯ

Суть метода проста. Используя интерфейс TakesScreenshot, просим драйвер сделать скриншот и записать результат во временный файл. Подобное приведение типов уже встречалось, когда мы исполняли JavaScript-код. Далее создаём файл с определенным именем в конкретной папке и копируем содержимое временного файла (со скриншотом) в файл на диске.

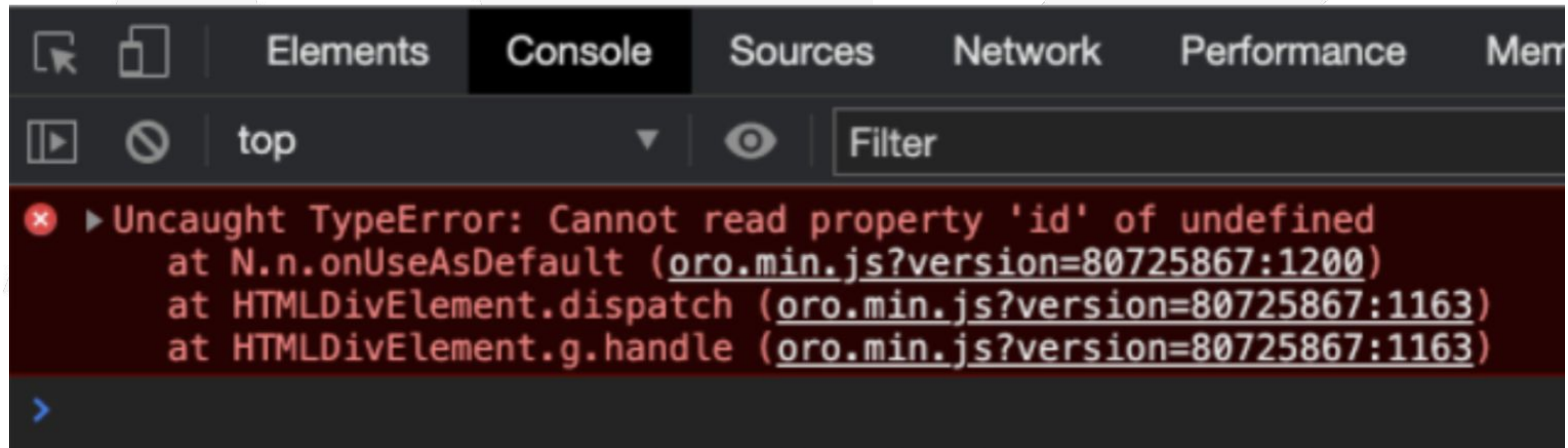
Используем в методе «слушателя»:

```
@Override
public void onException(Throwable throwable, WebDriver
driver) {
    String fileName = "failure-" +
System.currentTimeMillis() + ".png";
    ScreenshotMaker.makeScreenshot(driver, fileName);
    logger.error("onException: Screenshot saved in
target/" + fileName);
}
```


Логи браузера



В процессе ручного тестирования внимательные инженеры обращают внимание на ошибки в консоли браузера, так как появление этих ошибок свидетельствует о некорректно работающей логике.



Логи браузера



В рамках автоматизированного тестирования, разумеется, нет возможности открывать консоль и смотреть на её состояние. Тесты не запускаются на локальных машинах, но возможность получения логов предоставляет WebDriver.

`@AfterEach`

```
public void tearDown() {  
    // вывод всех ошибок браузера после каждого теста  
    LogEntries browserLogs =  
driver.manage().logs().get(LogType.BROWSER);  
    List<LogEntry> allLogRows = browserLogs.getAll();  
    if (allLogRows.size() > 0 ) { // Обработка ситуации  
    }  
  
    // -----  
    driver.quit();  
}
```

Рассмотрим код построчно:

- Драйвер возвращает браузерные логи в виде объекта `LogEntries` — имплементирующий интерфейс `Iterable`.
- Получение объекта списка логов.
- Если размер списка отличается от нуля, предпринимаем какие-нибудь действия, например, создаём файл с браузерными ошибками.

Практическое задание



Необходимо взять тест из практического задания в теме 17 и добавить Allure-репорт.